

Adding features: Validation Test Mode

Context information

- Node 0: run with `./build/bin/bitcoind`, `wsl`, arguments in file `cli.py` line 41.
- Node 1: run with `bitcoind`, `windows`, arguments in file `cli.py` line 42.
- Node 2: run with `bitcoind`, `windows`, arguments in file `cli.py` line 43.
- Node 3: run with `bitcoind`, `windows`, arguments in file `cli.py` line 44.
- Node 4: run with `bitcoind`, `windows`, arguments in file `cli.py` line 45.
- Node 5: run with `bitcoind`, `windows`, arguments in file `cli.py` line 46.
- Node 6: run with `./build/bin/bitcoind`, `wsl`, arguments in file `cli.py` line 47.
- Peers: for a node let's call `peers_of_node[i]` be the list of peers of node number `i`:
 - For example: `peers_of_node[3] = [node i, node j, node k]`. Then nodes `i`, `j`, `k` are the nodes that directly fully connect to node 3, i.e. the connections between them are full connections that can relay both blocks and transactions (not a block-only-connection).

Phase 1: Create groundtruth

Scope and output

- Read `peers_of_node[i]` for every $i \in \{1, 2, 3, 4, 5\}$.
- Nodes 0 and 6 are excluded from the Validation Test Mode ground truth.
- Keep only full-relay peers: while reading `peers_of_node[i]`, exclude peers whose `getpeerinfo.connection_type` is `block-relay-only`, `addr-fetch`, or `feeler`.
- Besides nodes 1--5, include every **external peer node**: a node whose ID is not in $\{0, 1, 2, 3, 4, 5, 6\}$ and which occurs in at least one of `peers_of_node[1]` through `peers_of_node[5]`.
- Exclude an external peer that cannot be connected to in the reverse direction. In `getpeerinfo`, these are peers whose `addr` is a localhost endpoint (`127.0.0.1:xxxx`) or an IPv6 endpoint (for example, `[2001:db8::1]:xxxx`).
- Do **not** require an external peer to occur in every peer list. It is included when it occurs in **any one** of those lists.

Let the external ground-truth vertices be:

$$V_{external}^{before} = \left(\bigcup_{i=1}^5 \text{peers_of_node}[i] \right) \setminus (\{0, 1, 2, 3, 4, 5, 6\} \cup V_{unconnectable} \cup V_{non_full_relay}),$$

$$V_{unconnectable} = \{v \mid \text{addr}(v) \text{ is } 127.0.0.1:\text{port} \text{ or is an IPv6 endpoint}\},$$

$$V_{non_full_relay} = \{v \mid \text{connection_type}(v) \in \{\text{block-relay-only, addr-fetch, feeler}\}\}$$

and the complete Phase 1 vertex set be:

$$S_{groundtruth}^{before} = \{1, 2, 3, 4, 5\} \cup V_{external}^{before}$$

Output the before-groundtruth edges $E_{groundtruth}^{before}$ as directed pairs. For every observed connection, include **both** (u, v) and (v, u) ; do not deduplicate them into one canonical undirected pair. Only connections observed from a node in $\{1, 2, 3, 4, 5\}$ are required in this phase:

$$E_{groundtruth}^{before} = \{$$

$$\begin{array}{l} (u, v) \quad \quad \quad | \\ u \in \{1, 2, 3, 4, 5\} \quad \wedge v \in \text{peers_of_node}[u] \quad \wedge \\ v \in S_{groundtruth}^{before} \\ \} \cup \{ \\ (v, u) \quad \quad \quad | \\ u \in \{1, 2, 3, 4, 5\} \quad \wedge v \in \text{peers_of_node}[u] \quad \wedge \\ v \in S_{groundtruth}^{before} \\ \} \end{array}$$

For an external peer, use a stable network identity (for example its normalized `addr_endpoint`) so that the same peer occurring in several lists is one vertex. The per-connection RPC `peer_id` must not be used as this identity, because it is local to the observing node.

Phase 2: INVBLOCK Filter

Node 0, 6 setup:

- Connect nodes 0, 6 to all the nodes in $S_{groundtruth}^{before}$.
- Note: `MAX_ADDNODE_CONNECTIONS` in `src/net.h` is increased from 8 to 125 to allow node 0 and node 6 to establish manual outbound connections to all targets in $S_{groundtruth}^{before}$ simultaneously.

- Separate log outputs: Node 0 outputs logging to `txprobe_0.log` and Node 6 outputs logging to `txprobe_6.log` (via `-txprobelogfile` configuration).

Workflow

1. Clean up and establish the filter links.

Before initiating new connections to $S_{groundtruth}^{before}$, perform cleanup on nodes 0 and 6:

- Inspect existing `getaddednodeinfo` : for any node not in $S_{groundtruth}^{before}$, execute `addnode <addr> remove` .
- Inspect active `getpeerinfo` : for any connected peer not in $S_{groundtruth}^{before}$, execute `disconnectnode <addr>` .
- For every target $v \in S_{groundtruth}^{before}$, if v is not yet in `getaddednodeinfo` , node 0 and node 6 each initiate and keep one persistent manual connection using `addnode <addr> add` . Use the exact Phase 1 `addr` , including its port. Do not remove any of these connections after the experiment.

2. **Wait for all links.** Poll `getpeerinfo` on nodes 0 and 6 until each node has a qualifying full-relay connection to every target. On a polling timeout, print `timeout` , reset the elapsed counter to zero, and continue waiting.

3. **First inventory announcement.** Node 0 creates one transaction itx and sends an `INV(itx)` message to every node in $S_{groundtruth}^{before}$.

4. **Second inventory announcement.** After approximately 30 seconds, node 6 sends `INV(itx)` to every node in $S_{groundtruth}^{before}$.

5. **Collect the INVBLOCK failures.** Inspect `txprobe_6.log` of node 6. If a node $v \in S_{groundtruth}^{before}$ responds to node 6 with `getdata(itx)` , add v to the set of nodes that cannot be INVBLOCKed:

$$S_{noinvblock} = \{v \in S_{groundtruth}^{before} \mid v \text{ sent } \text{getdata}(itx) \text{ to node 6}\}.$$

Nodes 0 and 6 are instrumentation nodes: their Phase 2 links are not vertices or edges of the topology being inferred. Phase 2 leaves $S_{groundtruth}^{before}$ and $E_{groundtruth}^{before}$ unchanged.

Phase 3: Crafting real TxProbe's transaction

Phase 4: INVBLOCK (n + 1) conflicting transactions

- Retrieve nodes (in $\{1,2,3,4,5\}$) that did not receive correct transactions. Eliminating them.

Phase 5: Perform the sending transactions phase of TxProbe

- Sending nodes $S_{groundtruth}^{before} \setminus \{1, 2, 3, 4, 5\}$ the flooding transaction ftx
- Sending $\{1, 2, 3, 4, 5\}$ transactions ptx_1, ptx_2, ptx_3, ptx_4, ptx_5
- Sending markers to $\{1, 2, 3, 4, 5\}$ with corresponding transactions mtx_1, mtx_2, mtx_3, mtx_4, mtx_5.

Phase 6: Retrieve nodes that know transactions that they are not supposed to know or nodes that unknown transactions that they are supposed to know

- First we sending getrawtransaction messages about all $(n + 1)$ conflicting transactions ptx_1, ptx_2, ..., ptx_n, ftx.
- We need to retrieve the wrong-tx nodes - S_{wrong} , the set of nodes knowing/unknowing transactions that they aren't supposed to know. Use the getrawtransaction RPC command:

$$S_{wrong} = \left\{ \begin{array}{ll} u & | \\ u \text{ knows ftx} & \vee \\ (u \text{ knows ptx}_v \wedge v \neq u) & \vee \\ u \text{ doesn't know ptx}_u & \\ \end{array} \right\}$$

Phase 7: Sending inventories about all markers to all nodes in $S_{groundtruth}^{before}$.

- If a node i requests getdata(mtx_j) then we conclude that node does not have direct connection to node j.
- Output:

$$nE = \{ (u, v) \mid (u \in \{1, 2, 3, 4, 5\} \wedge v \in S_{groundtruth}^{before} \wedge v \text{ requested } getdata(mtx_u)) \vee (v \in \{1, 2, 3, 4, 5\} \wedge u \in S_{groundtruth}^{before} \wedge u \text{ requested } getdata(mtx_v)) \}$$

- For each node in $\{1, 2, 3, 4, 5\}$, send the `getrawtransaction` of all the marker transactions `mtx_1`, `mtx_2`, ..., `mtx_n`. Update S_{wrong} :

$$S_{wrong} = S_{wrong} \cup \{ u \mid u \text{ knows } mtx_v \wedge u \neq v \}$$

Phase 8: Transitory filter

- Update: `peers_of_node[i]` $\forall i \in [0..5]$. For each node, retrieve the current list of peers and update it.
- Retrieve new groundtruth vertex set $S_{groundtruth}^{after}$:
 - Initially: $S_{groundtruth}^{after} = S_{groundtruth}^{before}$
 - For each node n_i in $S_{groundtruth}^{before}$, check 0 is still connect to n_i or not, i.e. $(n_i \in \text{peers_of_node}[0])$. If they are not connected, eliminate n_i from $S_{groundtruth}^{after}$.
- Retrieve after groundtruth edges $E_{groundtruth}^{after}$:

$$E_{groundtruth}^{after} = \{ (u, v) \mid (u \in \{1, 2, 3, 4, 5\} \wedge v \in \text{peers_of_node}[u]) \vee (v \in \{1, 2, 3, 4, 5\} \wedge u \in \text{peers_of_node}[v]) \}$$

- Retrieve iE the ignoring edges:

$$\begin{aligned}
iE = & \{ \\
& (u, v) \quad | \\
& ((u, v) \in E_{groundtruth}^{before} \quad \wedge (u, v) \notin E_{groundtruth}^{after}) \quad \vee \\
& ((u, v) \notin E_{groundtruth}^{before} \quad \wedge (u, v) \in E_{groundtruth}^{after}) \quad \vee \\
& (u \in S_{groundtruth}^{before} \setminus \{1, 2, 3, 4, 5\} \quad \wedge v \in S_{groundtruth}^{before} \setminus \{1, 2, 3, 4, 5\}) \quad \vee \\
& (u \in (S_{groundtruth}^{before} \setminus S_{groundtruth}^{after}) \cup S_{wrong}) \quad \vee \\
& (v \in (S_{groundtruth}^{before} \setminus S_{groundtruth}^{after}) \cup S_{wrong}) \\
& \}
\end{aligned}$$

Phase 9: Connection Inference

- Define the first matrix - groundtruth matrix **gt**:

$$\mathbf{gt} = \begin{bmatrix} gt_{11} & gt_{12} & \cdots & gt_{1n} \\ gt_{21} & gt_{22} & \cdots & gt_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ gt_{n1} & gt_{n2} & gt_{n3} & gt_{nn} \end{bmatrix}$$

$$\begin{aligned}
\text{Where: } \mathbf{gt}_{ij} &= * \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \in iE \\
\mathbf{gt}_{ij} &= 0 \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \notin E_{groundtruth}^{after} \setminus iE \\
\mathbf{gt}_{ij} &= 1 \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \in E_{groundtruth}^{after} \setminus iE
\end{aligned}$$

- Define the second matrix - inference matrix **infer**:

$$\mathbf{infer} = \begin{bmatrix} infer_{11} & infer_{12} & \cdots & infer_{1n} \\ infer_{21} & infer_{22} & \cdots & infer_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ infer_{n1} & infer_{n2} & infer_{n3} & infer_{nn} \end{bmatrix}$$

$$\begin{aligned}
\text{Where: } \mathbf{infer}_{ij} &= * \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \in iE \\
\mathbf{infer}_{ij} &= 0 \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \in nE \setminus iE \\
\mathbf{infer}_{ij} &= 1 \text{ if } (S_{groundtruth}^{before}[i], S_{groundtruth}^{before}[j]) \in E_{groundtruth}^{after} \setminus (iE \cup nE)
\end{aligned}$$